Code_Documentation

Hasanhejleh

2025-03-12

Contents

1 Divvy Bike Share Data Analysis

1.1 Project Overview

This analysis explores the Divvy bike share dataset to identify trends in bike usage, membership types, and station popularity. The dataset includes trip data from January 2024 to January 2025, with details such as trip duration, distance, start/end stations, and user type (casual vs. annual members).

1.2 Setting Up The Environment

1.2.1 1. Install&Load Required Packages.

Before starting the analysis, ensure the necessary R packages are installed. These packages include `tidyverse`, `skimr`, `janitor`, `dplyr`, and `geosphere`.

```
#install.packages("tidyverse")
#install.packages("skimr")
#install.packages("janitor")
#install.packages("dplyr")
#install.packages("geosphere")
#install.packages("tinytex")
#tinytex::install_tinytex()
```

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.4.2
```

```
## Warning: package 'dplyr' was built under R version 4.4.2
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(skimr)
```

```
## Warning: package 'skimr' was built under R version 4.4.2
```

```
library(janitor)
```

```
## Warning: package 'janitor' was built under R version 4.4.2
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##     chisq.test, fisher.test
```

```
library(dplyr)
library(geosphere)
```

```
## Warning: package 'geosphere' was built under R version 4.4.2
```

```
library(tinytex)
```

1.3 Loading and Combining Data

1.3.1 1. Load Monthly Trip Data

The Divvy trip data is stored in separate CSV files for each month. Load each file into a dataframe.

```
divvy2024_01_df <- read_csv("202401-divvy-tripdata.csv")
```

```
## Rows: 144873 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
divvy2024_02_df <- read_csv("202402-divvy-tripdata.csv")
```

```
## Rows: 223164 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
divvy2024_03_df <- read_csv("202403-divvy-tripdata.csv")
```

```
## Rows: 301687 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
```

```
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
divvy2024_04_df <- read_csv("202404-divvy-tripdata.csv")
```

```
## Rows: 415025 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
divvy2024_05_df <- read_csv("202405-divvy-tripdata.csv")
```

```
## Rows: 609493 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
divvy2024_06_df <- read_csv("202406-divvy-tripdata.csv")
```

```
## Rows: 710721 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
divvy2024_07_df <- read_csv("202407-divvy-tripdata.csv")
```

```
## Rows: 748962 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
divvy2024_08_df <- read_csv("202408-divvy-tripdata.csv")
```

```
## Rows: 755639 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
divvy2024_09_df <- read_csv("202409-divvy-tripdata.csv")
```

```
## Rows: 821276 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
divvy2024_10_df <- read_csv("202410-divvy-tripdata.csv")
```

```
## Rows: 616281 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
divvy2024_11_df <- read_csv("202411-divvy-tripdata.csv")
```

```
## Rows: 335075 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
divvy2024_12_df <- read_csv("202412-divvy-tripdata.csv")
```

```
## Rows: 178372 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
divvy2025_01_df <- read_csv("202501-divvy-tripdata.csv")
```

```
## Rows: 138689 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

1.3.2 2. Check Data Integrity

Verify the structure and integrity of each dataframe by inspecting column names, the first few rows, and the data types.

-Repeat this step for all 13 dataframes.

```
colnames(divvy2024_01_df)
```

```
##  [1] "ride_id"            "rideable_type"      "started_at"
##  [4] "ended_at"           "start_station_name" "start_station_id"
##  [7] "end_station_name"   "end_station_id"     "start_lat"
## [10] "start_lng"          "end_lat"            "end_lng"
## [13] "member_casual"
```

```
head(divvy2024_01_df)
```

```
## # A tibble: 6 x 13
##   ride_id          rideable_type started_at          ended_at
##   <chr>            <chr>         <dttm>              <dttm>
## 1 C1D650626C8C899A electric_bike 2024-01-12 15:30:27 2024-01-12 15:37:59
## 2 EECD38BDB25BFCB0 electric_bike 2024-01-08 15:45:46 2024-01-08 15:52:59
## 3 F4A9CE78061F17F7 electric_bike 2024-01-27 12:27:19 2024-01-27 12:35:19
## 4 0A0D9E15EE50B171 classic_bike  2024-01-29 16:26:17 2024-01-29 16:56:06
## 5 33FFC9805E3EFF9A classic_bike  2024-01-31 05:43:23 2024-01-31 06:09:35
## 6 C96080812CD285C5 classic_bike  2024-01-07 11:21:24 2024-01-07 11:30:03
## # i 9 more variables: start_station_name <chr>, start_station_id <chr>,
## #   end_station_name <chr>, end_station_id <chr>, start_lat <dbl>,
## #   start_lng <dbl>, end_lat <dbl>, end_lng <dbl>, member_casual <chr>
```

```r
str(divvy2024_01_df)
```

```
## spc_tbl_ [144,873 x 13] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ ride_id           : chr [1:144873] "C1D650626C8C899A" "EECD38BDB25BFCB0" "F4A9CE78061F17F7" "0A0D9
##  $ rideable_type     : chr [1:144873] "electric_bike" "electric_bike" "electric_bike" "classic_bike"
##  $ started_at        : POSIXct[1:144873], format: "2024-01-12 15:30:27" "2024-01-08 15:45:46" ...
##  $ ended_at          : POSIXct[1:144873], format: "2024-01-12 15:37:59" "2024-01-08 15:52:59" ...
##  $ start_station_name: chr [1:144873] "Wells St & Elm St" "Wells St & Elm St" "Wells St & Elm St" "We
##  $ start_station_id  : chr [1:144873] "KA1504000135" "KA1504000135" "KA1504000135" "TA1305000030" ..
##  $ end_station_name  : chr [1:144873] "Kingsbury St & Kinzie St" "Kingsbury St & Kinzie St" "Kingsbu
##  $ end_station_id    : chr [1:144873] "KA1503000043" "KA1503000043" "KA1503000043" "13193" ...
##  $ start_lat         : num [1:144873] 41.9 41.9 41.9 41.9 41.9 ...
##  $ start_lng         : num [1:144873] -87.6 -87.6 -87.6 -87.6 -87.7 ...
##  $ end_lat           : num [1:144873] 41.9 41.9 41.9 41.9 41.9 ...
##  $ end_lng           : num [1:144873] -87.6 -87.6 -87.6 -87.6 -87.6 ...
##  $ member_casual     : chr [1:144873] "member" "member" "member" "member" ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   ride_id = col_character(),
##   ..   rideable_type = col_character(),
##   ..   started_at = col_datetime(format = ""),
##   ..   ended_at = col_datetime(format = ""),
##   ..   start_station_name = col_character(),
##   ..   start_station_id = col_character(),
##   ..   end_station_name = col_character(),
##   ..   end_station_id = col_character(),
##   ..   start_lat = col_double(),
##   ..   start_lng = col_double(),
##   ..   end_lat = col_double(),
##   ..   end_lng = col_double(),
##   ..   member_casual = col_character()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

1.3.3 3. Combine Data into a Single Dataframe

Aggregate all monthly data into a single dataframe for easier analysis.

-Note: change the path to the file you have the data inside.

```r
csv_files <- list.files(path = "D:/Moved Data/Google Data Analytics Program/Case Study Stuff/R Studio c
                        pattern = "*.csv",
                        full.names = TRUE)

divvy_combined_df <- csv_files %>%
  lapply(function(file) {

    df <- read_csv(file)
    df$yearmonth <- gsub(pattern = "[^0-9]", replacement = "",
                         basename(file))

    return(df)
```

```
  }) %>%
  bind_rows()
```

```
## Rows: 144873 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 223164 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 301687 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 415025 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 609493 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 710721 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
```

```
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 748962 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 755639 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 821276 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 616281 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 335075 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 178372 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
```

```
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
## Rows: 138689 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): ride_id, rideable_type, start_station_name, start_station_id, end_...
## dbl  (4): start_lat, start_lng, end_lat, end_lng
## dttm (2): started_at, ended_at
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

1.4 Data Cleaning and Pre processing

1.4.1 Data Cleaning Steps

1. **Reformat Columns**:
   - Convert `start_lat` and `end_lat` to numeric.
   - Convert `start_lng` and `end_lng` to numeric.
2. **Handle Missing Values**:
   - Remove rows with missing `end_lat` and `end_lng`.
   - Replace missing station names and IDs with "Unknown".
3. **Create New Columns**:
   - Calculate trip duration.
   - Calculate trip distance using the Haversine formula.
4. **Filter Invalid Trips**:
   - Remove trips with duration < 1.3 minutes.
   - Remove trips with distance < 200 meters or > 768 kilometers.
5. **Remove Duplicates**:
   - Round timestamps to the nearest minute.
   - Remove duplicate rows based on `ride_id`, `started_at`, and `ended_at`.

1.4.2 1. Reformat Columns

Ensure latitude and longitude columns are in the correct numeric format.

```
divvy_combined_df$start_lat <- as.double(divvy_combined_df$start_lat)
divvy_combined_df$end_lat <- as.double(divvy_combined_df$end_lat)
divvy_combined_df$start_lng <- as.double(divvy_combined_df$start_lng)
divvy_combined_df$end_lng <- as.double(divvy_combined_df$end_lng)
```

1.4.3 2. Handle Missing Values

Check for and address missing values in the dataset.

```r
colSums(is.na(divvy_combined_df))
```

```
##            ride_id      rideable_type         started_at           ended_at
##                  0                  0                  0                  0
## start_station_name   start_station_id   end_station_name     end_station_id
##            1096803            1096803            1128726            1128726
##          start_lat          start_lng            end_lat            end_lng
##                  0                  0               7293               7293
##      member_casual          yearmonth
##                  0                  0
```

```r
# Remove NA values for end_lat and end_lng
divvy_combined_df <- divvy_combined_df %>%
  filter(!is.na(end_lat))

# Replace NA with "Unknown" for station names and IDs
divvy_combined_df$start_station_name[is.na(divvy_combined_df$start_station_name)] <- "Unknown"
divvy_combined_df$start_station_id[is.na(divvy_combined_df$start_station_id)] <- "Unknown"
divvy_combined_df$end_station_name[is.na(divvy_combined_df$end_station_name)] <- "Unknown"
divvy_combined_df$end_station_id[is.na(divvy_combined_df$end_station_id)] <- "Unknown"

# Recheck for NA values
colSums(is.na(divvy_combined_df))
```

```
##            ride_id      rideable_type         started_at           ended_at
##                  0                  0                  0                  0
## start_station_name   start_station_id   end_station_name     end_station_id
##                  0                  0                  0                  0
##          start_lat          start_lng            end_lat            end_lng
##                  0                  0                  0                  0
##      member_casual          yearmonth
##                  0                  0
```

1.4.4 3. Create New Columns

- Create Trip Duration Column

Calculate the duration of each trip in minutes.

```r
divvy_combined_df$started_at <- as.POSIXct(divvy_combined_df$started_at, format = "%Y-%m-%d %H:%M:%S",
divvy_combined_df$ended_at <- as.POSIXct(divvy_combined_df$ended_at, format = "%Y-%m-%d %H:%M:%S", tz =

duration <- difftime(divvy_combined_df$ended_at, divvy_combined_df$started_at, units = "mins")
divvy_combined_df <- divvy_combined_df %>% mutate(duration)
```

- Create Trip Distance Column

Calculate the distance of each trip using the Haversine formula.

```r
divvy_combined_df <- divvy_combined_df %>%
  mutate(distance = distHaversine(cbind(start_lng, start_lat), cbind(end_lng, end_lat)))
```

- Create Weekday Column

```r
# Create new start_weekday column

divvy_combined_df <- divvy_combined_df %>%
  mutate(start_weekday = weekdays(started_at))
```

1.4.5 4. Filter Invalid Trips

Remove trips with unrealistic durations or distances, and calculate the percentage of those trips.

```r
percent_short_trips <- mean(divvy_combined_df$distance < 200 | divvy_combined_df$duration < 1.30 | divvy
print(paste("Percentage of trips < 200 meters OR < 1.30 minutes OR > 768000 meters:", round(percent_shor
```

```
## [1] "Percentage of trips < 200 meters OR < 1.30 minutes OR > 768000 meters: 7.46 %"
```

```r
invalid_trips <- divvy_combined_df %>%
  filter(duration < 1.30 & (distance < 200 | distance > 768000))

divvy_combined_df <- divvy_combined_df %>%
  filter(duration >= 1.3 & distance >= 200 & distance <= 768000)

# Recheck

percent_short_trips <- mean(divvy_combined_df$distance < 200 | divvy_combined_df$duration < 1.30 | divvy
print(paste("Percentage of trips < 200 meters OR < 1.30 minutes OR > 768000 meters:", round(percent_shor
```

```
## [1] "Percentage of trips < 200 meters OR < 1.30 minutes OR > 768000 meters: 0 %"
```

1.4.6 5. Remove Duplicates

Identify and remove duplicate entries based on ride_id, started_at, and ended_at.

```r
duplicates <- divvy_combined_df %>%
  group_by(ride_id) %>%
  filter(n() > 1) %>%
  arrange(ride_id)

divvy_combined_df$started_at <- round_date(divvy_combined_df$started_at, unit = "minute")
divvy_combined_df$ended_at <- round_date(divvy_combined_df$ended_at, unit = "minute")

divvy_combined_df <- divvy_combined_df %>% distinct(ride_id, started_at, ended_at, .keep_all = TRUE)
```

1.5 Data Analysis and Visualization

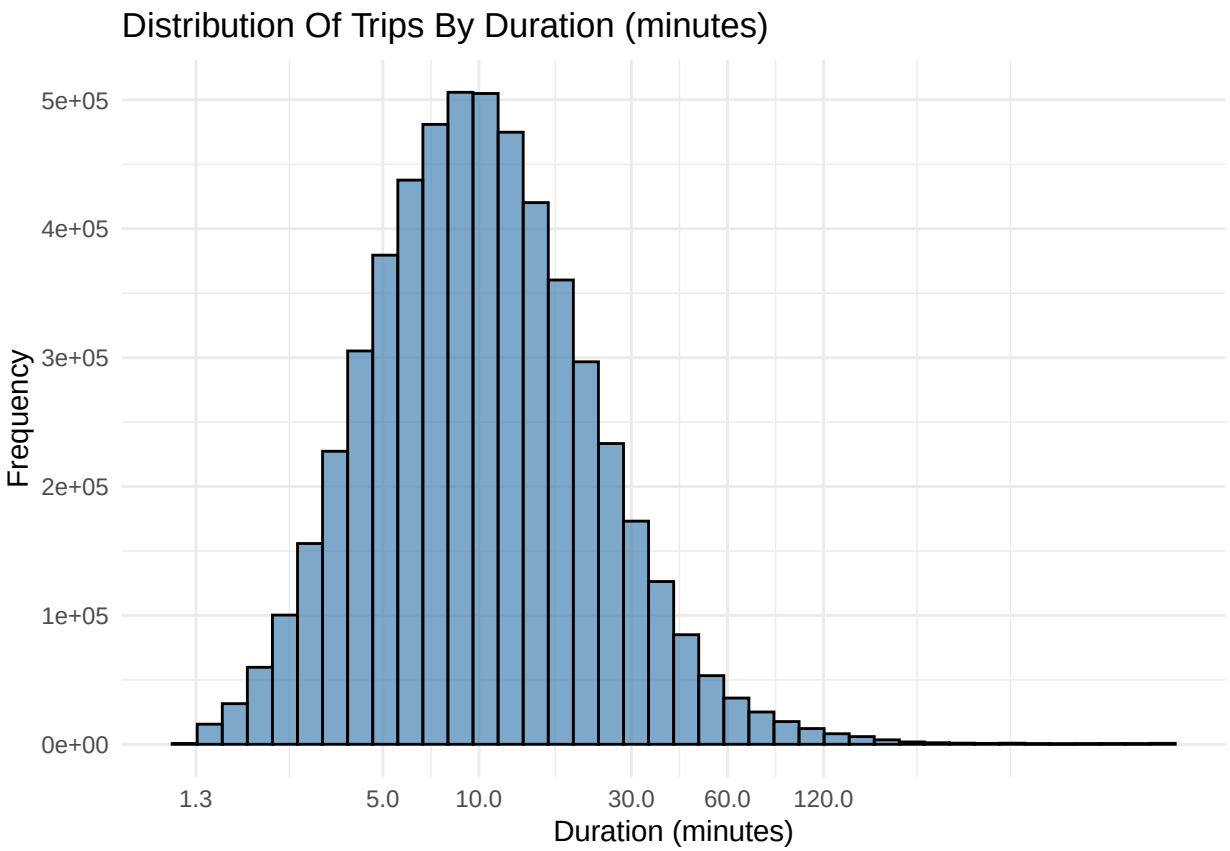
1.5.1 1. Distribution of Trip Durations

Analyze the distribution of trip durations and visualize it.

```
# Reformat duration column from timediff object to numeric
divvy_combined_df$duration <- as.numeric(divvy_combined_df$duration)

# Define bins bounds
lower_bound <- min(divvy_combined_df$duration)
upper_bound <- max(divvy_combined_df$duration)

# Define duration bins (adjust based on data distribution)
bins <- c(lower_bound, 5, 10, 30, 60, 120, 300, 600, 900, 1200, upper_bound)


# Compute percentage in each bin
duration_distribution <- divvy_combined_df %>%
  mutate(duration_bin = cut(duration, breaks = bins, include.lowest = TRUE)) %>%
  group_by(duration_bin) %>%
  summarise(count = n()) %>%
  mutate(percentage = (count / sum(count)) * 100) %>%
  arrange(desc(percentage))  # Sort by highest concentration

# View the distribution
print(duration_distribution)
```

```
## # A tibble: 10 x 3
##    duration_bin        count percentage
##    <fct>               <int>      <dbl>
##  1 (10,30]           2228916    40.2
##  2 (5,10]            1779199    32.1
##  3 [1.3,5]           1042149    18.8
##  4 (30,60]            383928     6.92
##  5 (60,120]            84530     1.52
##  6 (120,300]           20818     0.375
##  7 (300,600]            2773     0.0500
##  8 (600,900]            1021     0.0184
##  9 (1.2e+03,1.5e+03]     836     0.0151
## 10 (900,1.2e+03]         827     0.0149
```

```
# Visualize the distribution

ggplot(data = divvy_combined_df, aes(x = duration)) +
  geom_histogram(bins = 40, color = "black", fill = "steelblue", alpha = 0.7) +
  scale_x_log10(breaks = c(1.3, 5, 10, 30, 60 ,120))+
  labs(
    title = "Distribution Of Trips By Duration (minutes)",
    x = "Duration (minutes)",
    y = "Frequency"

  ) +
  theme_minimal()
```

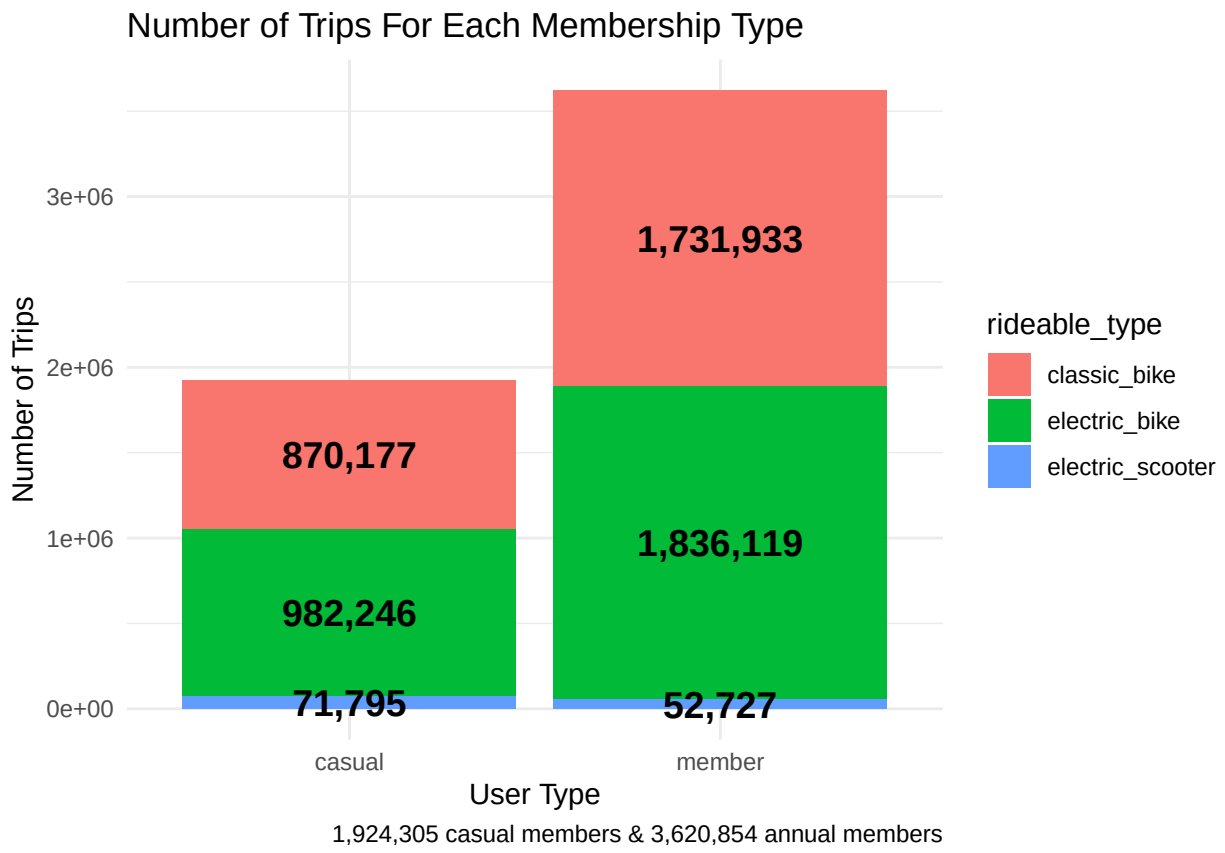


1.5.2 2. Membership Type & Number of Trips

Visualize the relationship between membership type and the number of trips.

```
# Group trips by membership type & bike type

avg_duration <- divvy_combined_df %>%
  group_by(member_casual, rideable_type) %>%
  summarise(average_duration = mean(duration), trip_count = n(), .groups = "drop") %>%
  arrange(member_casual, rideable_type)

# Create a stacked bar chart visualization for the segments of the data

ggplot(avg_duration, aes(x = member_casual, y = trip_count, fill = rideable_type)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = scales::comma(trip_count)),
            position = position_stack(vjust = 0.5),
            color = "black", size = 5, fontface = "bold") +
  labs(
    title = "Number of Trips For Each Membership Type",
    x = "User Type",
    y = "Number of Trips",
    caption = "1,924,305 casual members & 3,620,854 annual members"
  ) +
  theme_minimal() +
```

```
  theme(plot.caption = element_text(hjust = 1))
```

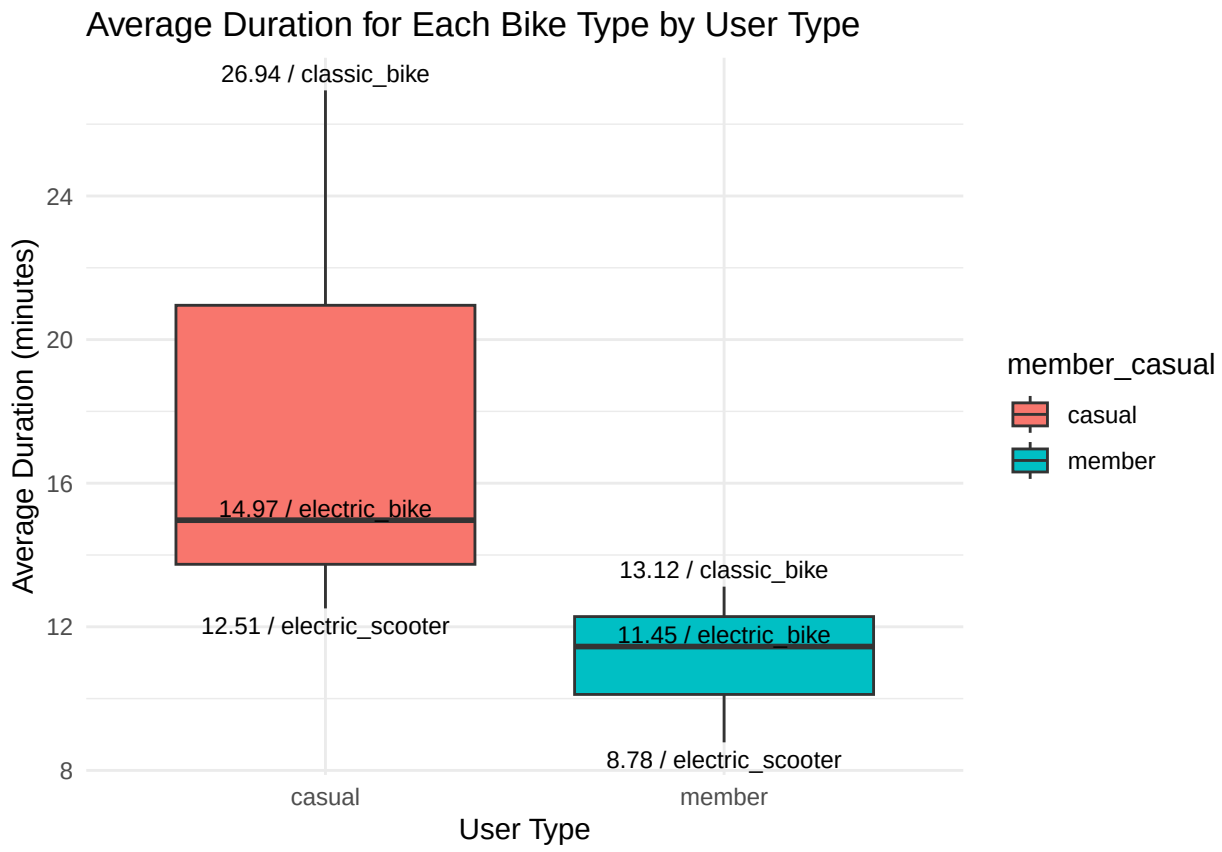


1.5.3 3. Average Duration for Each Bike Type by User Type

Analyze the average trip duration for each membership type.

```
# Precompute summary stats (min, max, median) for each member type

summary_data <- avg_duration %>%
  group_by(member_casual) %>%
  summarise(
    min_duration = min(average_duration),
    min_rideable = rideable_type[which.min(average_duration)],
    max_duration = max(average_duration),
    max_rideable = rideable_type[which.max(average_duration)],
    median_duration = median(average_duration),
    median_rideable = rideable_type[which.min(abs(average_duration - median(average_duration)))], # Clo
    .groups = "drop"
  )

# Create the boxplot
ggplot(avg_duration, aes(x = member_casual, y = average_duration, fill = member_casual)) +
  geom_boxplot() +
```

```
# Add min value text
geom_text(data = summary_data, aes(x = member_casual, y = min_duration,
                                   label = paste0(round(min_duration, 2), " / ", min_rideable)),
          vjust = 1.5, color = "black", size = 3) +

# Add max value text
geom_text(data = summary_data, aes(x = member_casual, y = max_duration,
                                   label = paste0(round(max_duration, 2), " / ", max_rideable)),
          vjust = -0.5, color = "black", size = 3) +

# Add median value text
geom_text(data = summary_data, aes(x = member_casual, y = median_duration,
                                   label = paste0(round(median_duration, 2), " / ", median_rideable)),
          vjust = -0.2, color = "black", size = 3) +

labs(title = "Average Duration for Each Bike Type by User Type",
     x = "User Type",
     y = "Average Duration (minutes)") +
theme_minimal()
```



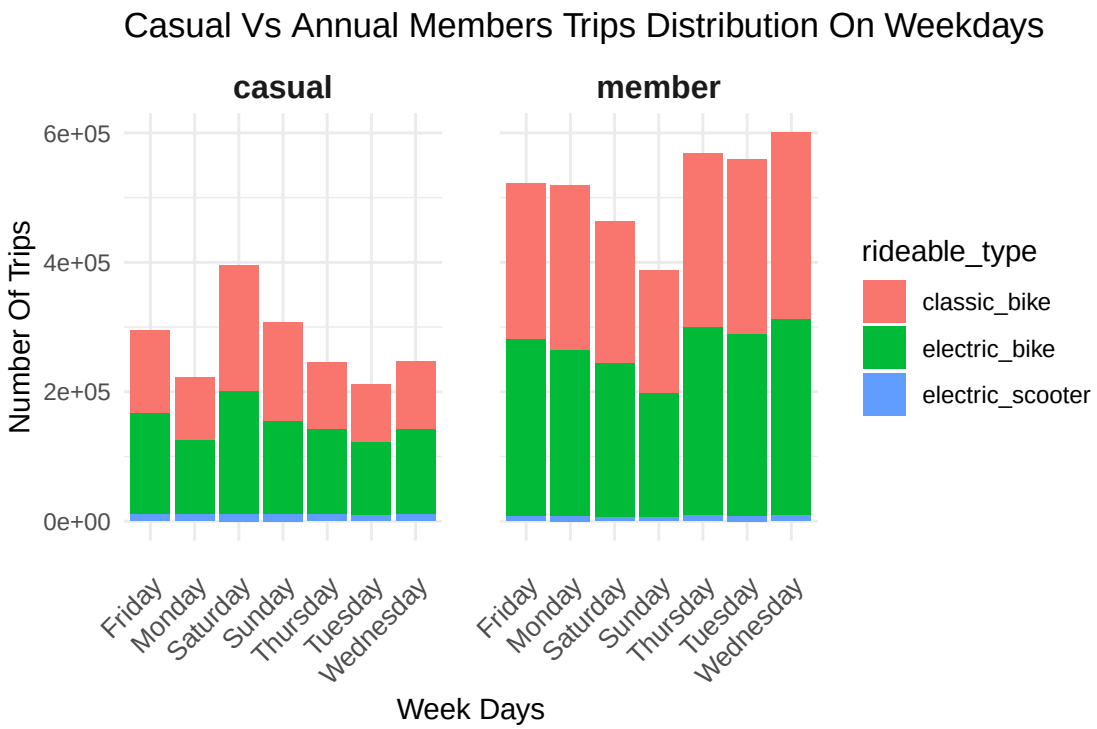
1.5.4 4. Casual Vs Annual Members Trips Distribution On Weekdays

Analyze which days of the week riders prefer to take trips.

```r
# Day of the week relationship & number of rides

divvy_combined_df %>%
  group_by(start_weekday) %>%
  summarise(num_of_rides = n()) %>%
  arrange(-num_of_rides)
```

```
## # A tibble: 7 x 2
##   start_weekday num_of_rides
##   <chr>                <int>
## 1 Saturday            859286
## 2 Wednesday           847131
## 3 Friday              818337
## 4 Thursday            812823
## 5 Tuesday             770828
## 6 Monday              741630
## 7 Sunday              694962
```

```r
# Visualize the relationship using a stacked bar chart with bike type segments

ggplot(data = divvy_combined_df, mapping = aes(x = start_weekday, fill = rideable_type)) +
  geom_bar() +
  facet_wrap(~member_casual) +
  labs(
    x = "Week Days",
    y = "Number Of Trips",
    title = "Casual Vs Annual Members Trips Distribution On Weekdays"
  ) +
  theme_minimal() +
  theme(
    # Add margin between facets
    panel.spacing = unit(1.5, "lines"),  # Increase spacing between facets

    # Adjust x-axis text angle, spacing, and alignment
    axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1, size = 10, margin = margin(t = 10)),

    # Add padding to the plot
    plot.margin = margin(1, 1, 1, 1, "cm"),  # Add margin around the plot

    # Adjust facet strip text
    strip.text = element_text(size = 12, face = "bold")  # Customize facet labels
  )
```



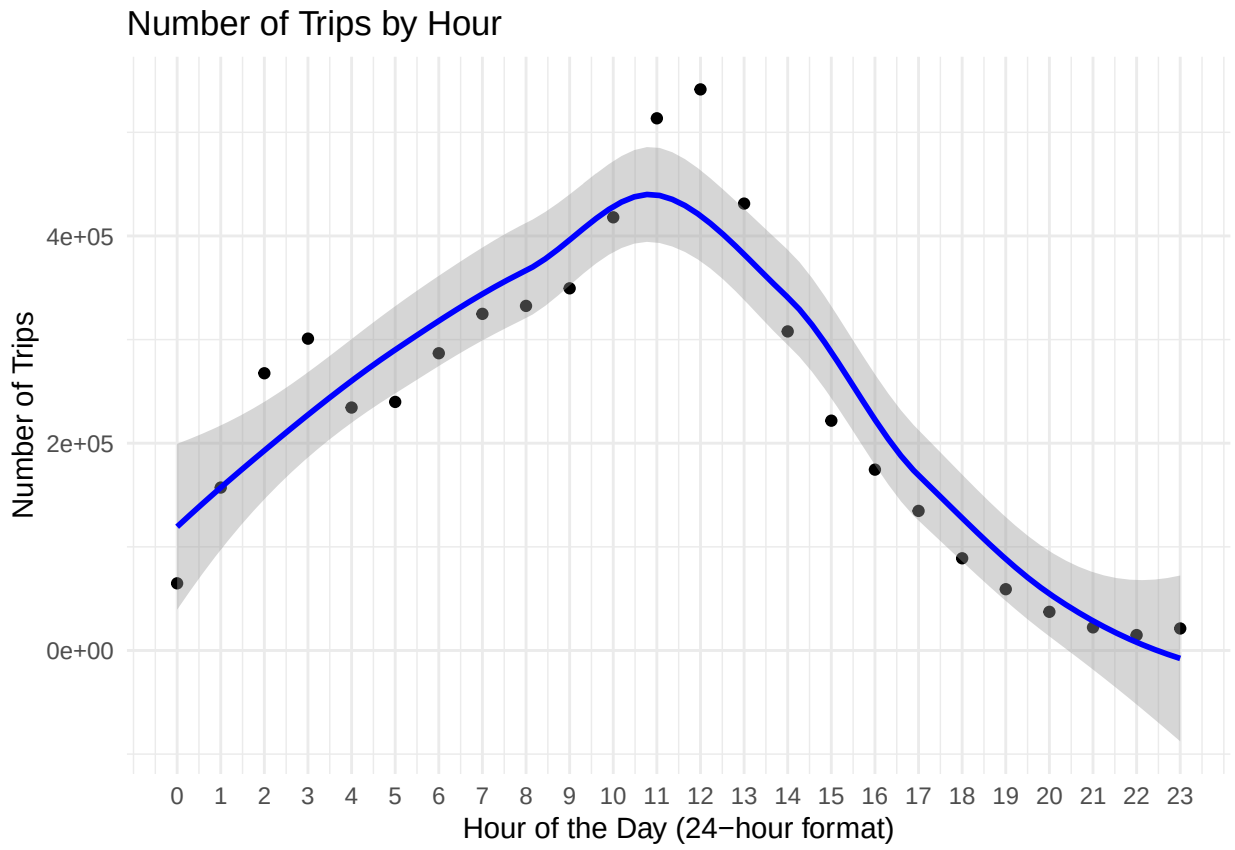
1.5.5 5. Number of Trips by Hour

Analyze when riders tend to use bikes throughout the day.

```
# Add a start_hour column by extracting the hour from started_at column

divvy_combined_df <- divvy_combined_df %>%
  mutate(start_hour = format(as.POSIXct(started_at), "%H"))

# Create a data frame to store the number of trips and their relative start hours

trips_by_hour <- divvy_combined_df %>%
  group_by(start_hour) %>%
  summarise(num_of_trips = n())

# Add a new formatted column for the start_hour values in the visualization

trips_by_hour <- trips_by_hour %>%
  mutate(start_hour_numeric = as.numeric(sub(":00:00", "", start_hour)))

ggplot(trips_by_hour, aes(x = start_hour_numeric, y = num_of_trips)) +
  geom_point() +
  geom_smooth(method = "loess", se = TRUE, color = "blue") +
  scale_x_continuous(breaks = seq(0, 23, by = 1), labels = paste0(seq(0, 23))) +
  labs(title = "Number of Trips by Hour",
```

```
       x = "Hour of the Day (24-hour format)",
       y = "Number of Trips") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



1.5.6 6. Busiest Stations by Membership Type

Identify and visualize the top 10 busiest stations for casual and annual members.

```
# Data Frame that includes: station names, membership type ,number of trips

start_station_counts <- divvy_combined_df %>%
  group_by(start_station_name, member_casual) %>%
  summarise(num_of_trips = n(), .groups = 'drop') %>%
  arrange(-num_of_trips)
```

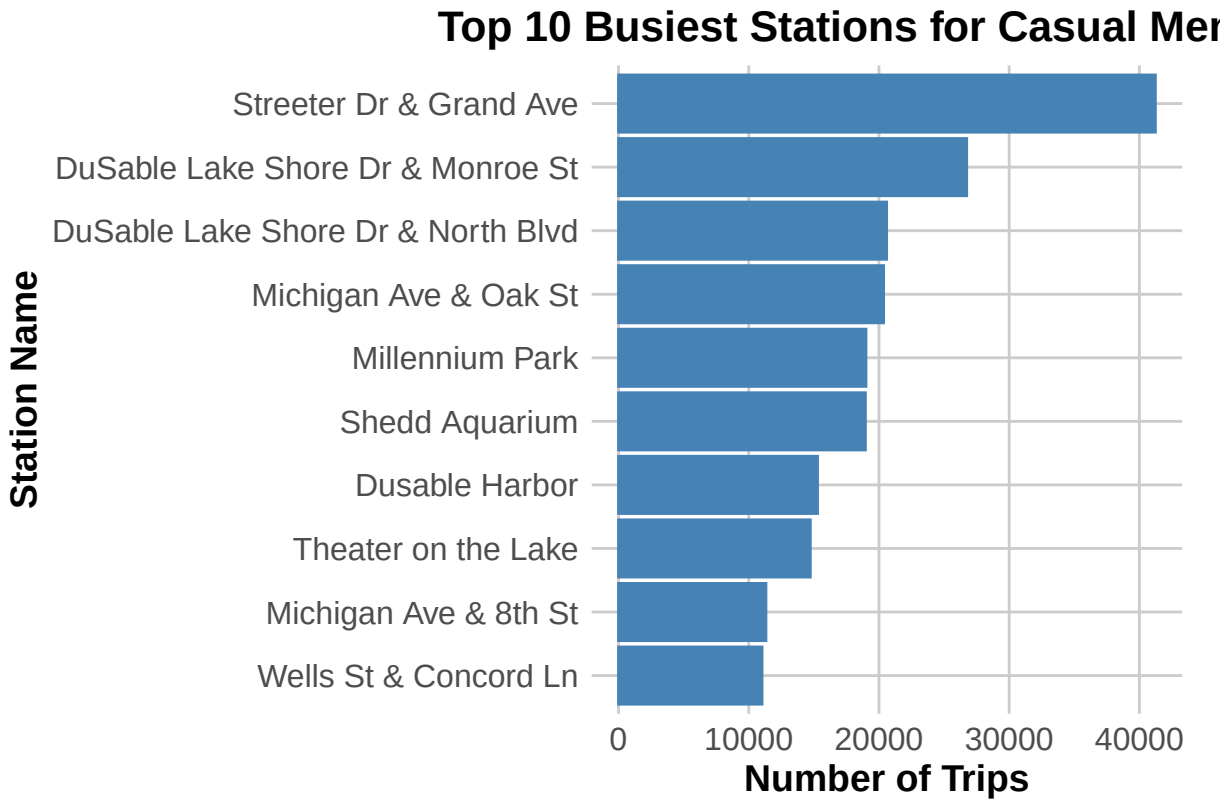
- Top 10 Busiest Casual stations Viz

```
# Filter stations by membership type since some stations were only used by 1 membership type

casual_stations <- start_station_counts %>%
  filter(member_casual == "casual", start_station_name != "Unknown") %>%
```

```
  arrange(-num_of_trips) %>%
  top_n(10)
```

```
## Selecting by num_of_trips
```

```
# Top 10 Busiest Stations for Casual Members Visualization

options(repr.plot.width = 10, repr.plot.height = 8)

ggplot(data = casual_stations, aes(x = reorder(start_station_name, num_of_trips), y = num_of_trips)) +
  geom_bar(stat = "identity", col = "steelblue", fill = "steelblue") +  # Color bars
  coord_flip() +  # Flip coordinates to make the bars horizontal
  labs(
    x = "Station Name",
    y = "Number of Trips",
    title = "Top 10 Busiest Stations for Casual Members"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(size = 12, family = "sans"),  # Improve x-axis text
    axis.text.y = element_text(size = 12, family = "sans"),  # Improve y-axis text
    axis.title.x = element_text(size = 14, face = "bold", family = "sans"),  # Improve x-axis title
    axis.title.y = element_text(size = 14, face = "bold", family = "sans"),  # Improve y-axis title
    plot.title = element_text(size = 16, face = "bold", family = "sans", hjust = 0.5),  # Improve plot
    panel.grid.major = element_line(color = "gray80"),  # Add light grid lines for readability
    panel.grid.minor = element_blank(), # Remove minor grid lines
    plot.margin = unit(c(0.5, 0.5, 0.5, 0.5), "cm")  # Adjust margins
  )
```



- Top 10 Busiest Annual stations Viz

```
# Filter stations by membership type since some stations were only used by 1 membership type

member_stations <- start_station_counts %>%
  filter(member_casual == "member", start_station_name != "Unknown") %>%
  arrange(-num_of_trips) %>%
  top_n(10)
```

```
## Selecting by num_of_trips
```

```
# Top 10 Busiest Stations for Annual Members Visualization

options(repr.plot.width = 10, repr.plot.height = 8)

ggplot(data = member_stations, aes(x = reorder(start_station_name, num_of_trips), y = num_of_trips)) +
  geom_bar(stat = "identity", col = "#ff7f0e", fill = "#ff7f0e") +  # Color bars
  coord_flip() +  # Flip coordinates to make the bars horizontal
  labs(
    x = "Station Name",
    y = "Number of Trips",
    title = "Top 10 Busiest Stations for Annual Members"
  ) +
  theme_minimal() +
  theme(
```

```
  axis.text.x = element_text(size = 12, family = "sans"),  # Improve x-axis text
  axis.text.y = element_text(size = 12, family = "sans"),  # Improve y-axis text
  axis.title.x = element_text(size = 14, face = "bold", family = "sans"),  # Improve x-axis title
  axis.title.y = element_text(size = 14, face = "bold", family = "sans"),  # Improve y-axis title
  plot.title = element_text(size = 16, face = "bold", family = "sans", hjust = 0.5),  # Improve plot
  panel.grid.major = element_line(color = "gray80"),  # Add light grid lines for readability
  panel.grid.minor = element_blank(), # Remove minor grid lines
  plot.margin = unit(c(0.5, 0.5, 0.5, 0.5), "cm")  # Adjust margins
)
```